Based on the **Project Context & Implementation Guide** for your final-year engineering project, you have completed **Phase 0 (Repository Setup & Stack Initialization)**.

The complete, phased **Plan of Action** below outlines the remaining steps to build the application successfully from end to end. Each phase maps directly to your project roadmap and ensures that core local functionality, computer vision, simulation, and incident recording are implemented reliably.

Complete Plan of Action: AI-Based Driver Safety Monitoring System

Phase 1: UI Shell & Navigation Layout

- **Goal:** Establish the multi-page layout and application shell using Next.js App Router, Tailwind CSS, and layout components.
- **Deliverables:**
    - Create routes: / (Home/Dashboard), /simulator, /incidents, /incidents/[id], and /settings.
    - Build a consistent navigation bar/sidebar and responsive container layout.
    - Add placeholder states for recent events, safety disclaimers, and system readiness indicators.
- **Manual Verification:** Navigate between all pages smoothly without errors or broken links.

Phase 2: Driving Simulator (Phaser 3)

- **Goal:** Implement the 2D driving simulation environment where users can control speed and trigger test events.
- **Deliverables:**
    - Create the Phaser canvas component with dynamic client-side loading (ssr: false).
    - Implement basic road movement, vehicle rendering, and a telemetry speed state (default limit: $20km/h$).
    - Build UI controls for accelerating, decelerating, and triggering simulated test events (e.g., overspeed or collision-like events).
- **Manual Verification:** Start the simulator, adjust speed past $20km/h$, and verify that the speedometer and UI state update correctly.

Phase 3: Camera Stream Integration

- **Goal:** Handle hardware camera permissions and live video preview safely within client components.
- **Deliverables:**
    - Build a camera management module utilizing navigator.mediaDevices.getUserMedia.

    - Create a React component for the live camera preview with explicit start/stop controls.
    - Handle permission denial and secure context errors gracefully without crashing the app.
- **Manual Verification:** Grant camera permission, confirm live video feed appears, and verify proper error handling if permission is denied.

Phase 4: Computer Vision Pipeline (MediaPipe Tasks Vision)

- **Goal:** Process camera frames on-device to track facial landmarks, eye state, and head orientation.
- **Deliverables:**
    - Integrate MediaPipe Face Landmarker for web.
    - Implement Eye Aspect Ratio (EAR) calculations to detect brief blinks versus sustained eye closure (drowsiness estimation).
    - Implement head pose estimation (yaw/pitch) to detect sustained driver distraction.
    - Handle "face not detected" and low-light/poor landmark edge cases gracefully.
- **Manual Verification:** Verify that the UI displays real-time face detection status, eye state, and head-pose indicators.

Phase 5: Safety Event Engine & Alert System

- **Goal:** Centralize real-time safety rules, cooldowns, and event emission.
- **Deliverables:**
    - Build a central event bus/safety engine that monitors speed rules, drowsiness intervals, and distraction timers.
    - Configure cooldowns and hysteresis to prevent alert spamming.
    - Implement visual warning panels with clear severity levels (info, warning, high).
- **Manual Verification:** Exceed the speed threshold or simulate prolonged eye closure to verify that alerts trigger correctly with appropriate cooldowns.

Phase 6: Local Database Layer (Dexie.js + IndexedDB)

- **Goal:** Enable reliable, local-first persistence for sessions, incident metadata, and records.
- **Deliverables:**
    - Define the Dexie database schema for sessions, incidents, and recordings.
    - Implement data access functions to create, read, update, and delete records locally.
    - Implement storage quota error handling.
- **Manual Verification:** Create test records and verify that they persist across page reloads.

Phase 7: Rolling Video Buffer & Incident Preservation

- **Goal:** Maintain a rolling video buffer of approximately 120 seconds using the MediaRecorder API and preserve clips on incident triggers.
- **Deliverables:**
    - Implement chunked video recording using supported MIME types (MediaRecorder.isTypeSupported).
    - Create a queue-based rolling buffer management system that discards segments older than 120 seconds unless retained.
    - Implement incident preservation logic to lock pre-event and post-event video blobs into IndexedDB upon an event trigger.
- **Manual Verification:** Let the buffer run for over 2 minutes, trigger a test incident, and verify that the preceding video window is successfully saved to IndexedDB.

Phase 8: Motion Sensor Module

- **Goal:** Integrate device motion and orientation APIs for experimental movement analysis with a simulation fallback.
- **Deliverables:**
    - Implement DeviceMotionEvent and DeviceOrientationEvent listeners with proper permission handling for mobile/supported devices.
    - Build a heuristic scoring algorithm to detect potential collision-like movement patterns.
    - Ensure a robust simulation fallback so features remain testable on desktop browsers without physical sensors.
- **Manual Verification:** Test sensor input where supported or trigger simulation-based collision events to confirm they flow into the incident pipeline.

Phase 9: Dashboard Integration & Incident History

- **Goal:** Connect all subsystems into a cohesive review workflow.
- **Deliverables:**
    - Build the Incident History page (/incidents) to list recorded events with timestamps, types, and speeds.
    - Build the Incident Detail view (/incidents/[id]) with pre-event video playback, sensor summaries, and deletion/export options.
- **Manual Verification:** Run a complete end-to-end simulation session, trigger an incident, review it in the history list, play back the saved video clip, and test deletion.

Phase 10: System Hardening & Documentation

- **Goal:** Validate performance, handle edge cases, and prepare final documentation for your project submission.
- **Deliverables:**
    - Conduct unit and integration testing across core algorithms (EAR, speed thresholds, rolling buffer retention).
    - Verify privacy behavior (ensuring no unauthorized cloud uploads or biometric template storage).

    - Finalize the README.md and project report metrics (false-positive rates, storage size, and browser limitations).
- **Manual Verification:** Run through the final acceptance checklist outlined in your project guide to ensure everything is fully functional.